

NumPy Library and Pandas Library

Table of Contents

1. Introduction to NumPy

- What is NumPy?3
- Why Use NumPy?.....3
- Which Language is NumPy written in?3

2. Getting Started with NumPy

- Installation of NumPy3
- Import NumPy3
- Checking NumPy Version3

3. NumPy Basics

- NumPy Data Types4
- Arrays.....4

4. Advanced NumPy Operations

- Linear Algebra5
- NumPy Random.....5
- Sorting and Searching6
- NumPy Mathematics.....6
- NumPy Statistics.....7
- NumPy Date Time8
- NumPy String.....8

5. Projects and Exercises

- Project 1: Data Analysis with NumPy9
- Exercise 2: Array Operations.....9
- Exercise 3: Linear Algebra Applications.....9
- Exercise 4: Random Number Generation9
- Exercise 5: Sorting and Searching9
- Exercise 6: Mathematical Computations9

6. Introduction To Pandas

- What is Pandas?10
- Why Use Pandas?10
- What can Pandas do?10

7. Pandas Getting Started

- Installation of Pandas 10
- Import Pandas 10
- Pandas as Pd..... 11
- Checking Pandas Version 11

8. Pandas Series

- What is a Series? 12
- Labels 12
- Key / Value object as series 12
- Data Frames 13

9. Pandas Data Frames

- What is a Data Frame? 13
- Creating Data Frames 13
- Row / Locate Row 14
- Named Indexes / Locate Named indexes 14

10. Pandas Read CSV

- Read CSV Files 15

11. Pandas Analyzing Data

- Viewing the Data 15
- Info About the Data 15
- Null Values 15
- Dealing With Rows and Columns..... 16
- Indexing and Selecting Data 16
- Working With Missing Data..... 18
- Iterating Over Rows and Columns..... 19

Introduction to NumPy

What is NumPy?

NumPy (Numerical Python) is a library in Python used for working with arrays. It also has functions for working in the domain of linear algebra, Fourier transform, and matrices.

Why Use NumPy?

NumPy provides a fast and efficient way to handle large arrays and matrices of numeric data. It includes mathematical functions and operations optimized for performance. It has faster computations capabilities compared to standard Python lists.

Which Language is NumPy written in?

The core of NumPy is written in C, which allows for high-performance computations.

Getting Started with NumPy

- Python: Ensure you have Python 3.6 or later installed. You can download it from the official Python website (<https://www.python.org/>).
- Additionally, you can also install the PyCharm IDE ([Installation Guide](#)).
- If you have Python and PIP already installed on a system, then installation of NumPy is very easy.

Installation of NumPy

```
python
```

```
pip install numpy
```

Copy code

Instruction: pip install numpy

Import NumPy

```
python
```

```
import numpy as np
```

Copy code

Instruction: import numpy as np

Checking NumPy Version

```
python
```

```
import numpy as np
print(np.__version__)
```

Copy code

Instruction: import numpy as np
print(np.__version__)

NumPy Basics

NumPy Data Types

Definition: NumPy supports various data types like int, float, complex, etc.

Example 1:

```
arr = np.array([1, 2, 3], dtype='int')
print(arr.dtype)
```

Example 2:

```
import numpy as np

# Create an array with integer data type

int_array = np.array([1, 2, 3], dtype='int')

print("Integer Array:", int_array)

print("Data Type:", int_array.dtype)

# Create an array with float data type

float_array = np.array([1.5, 2.5, 3.5], dtype='float')

print("Float Array:", float_array)

print("Data Type:", float_array.dtype)
```

Arrays

Definition: Arrays are central to NumPy and provide a means of storing and operating on homogeneous data.

Example 1:

```
arr = np.array([1, 2, 3, 4, 5])
print(arr)
```

Example 2:

```
import numpy as np

# Create a one-dimensional array

one_d_array = np.array([1, 2, 3, 4, 5])

print("One-dimensional Array:", one_d_array)
```

```
# Create a two-dimensional array
```

```
two_d_array = np.array([[1, 2, 3], [4, 5, 6]])  
print("Two-dimensional Array:\n", two_d_array)
```

Advanced NumPy Operations

Linear Algebra

Definition: NumPy provides functions to perform linear algebra operations like dot product, matrix multiplication, etc.

Example 1:

```
A = np.array([[1, 2], [3, 4]])  
B = np.array([[5, 6], [7, 8]])  
print(np.dot(A, B))
```

Example 2:

```
import numpy as np  
  
# Define two matrices  
A = np.array([[1, 2], [3, 4]])  
B = np.array([[5, 6], [7, 8]])  
  
# Perform matrix multiplication  
result = np.dot(A, B)  
print("Matrix A:\n", A)  
print("Matrix B:\n", B)  
print("Matrix Multiplication Result:\n", result)
```

NumPy Random

Definition: Functions to generate random numbers, arrays, and perform random sampling.

Example 1:

```
rand_array = np.random.rand(3, 3)  
print(rand_array)
```

Example 2:

```
import numpy as np

# Generate a 3x3 array of random numbers between 0 and 1

random_array = np.random.rand(3, 3)

print("Random Array:\n", random_array)
```

Sorting and Searching

Definition: NumPy provides efficient sorting and searching capabilities for arrays.

Example 1:

```
arr = np.array([3, 1, 2])
print(np.sort(arr))
```

Example 2:

```
import numpy as np

# Create an unsorted array

unsorted_array = np.array([3, 1, 2])

print("Unsorted Array:", unsorted_array)

# Sort the array

sorted_array = np.sort(unsorted_array)

print("Sorted Array:", sorted_array)

# Search for the index of element '2' in the sorted array

index = np.searchsorted(sorted_array, 2)

print("Index of element '2':", index)
```

NumPy Mathematics

Definition: Functions to perform mathematical operations on arrays.

Example 1:

```
arr = np.array([1, 2, 3])
print(np.sum(arr))
```

Example 2:

```
import numpy as np

# Create an array
array = np.array([1, 2, 3])

# Perform sum operation
sum_result = np.sum(array)
print("Sum:", sum_result)

# Perform mean operation
mean_result = np.mean(array)
print("Mean:", mean_result)
```

NumPy Statistics

Definition: Statistical functions such as mean, median, standard deviation, etc.

Example 1:

```
arr = np.array([1, 2, 3, 4, 5])
print(np.mean(arr))
```

Example 2:

```
import numpy as np

# Create an array
array = np.array([1, 2, 3, 4, 5])

# Calculate mean
mean = np.mean(array)
print("Mean:", mean)

# Calculate standard deviation
std_dev = np.std(array)
print("Standard Deviation:", std_dev)
```

NumPy Date Time

Definition: Functions to handle date and time-related operations.

Example 1:

```
dates = np.array(['2023-06-15', '2024-06-15'], dtype='datetime64')
print(dates)
```

Example 2:

```
import numpy as np

# Create an array of dates
date_array = np.array(['2023-06-15', '2024-06-15'], dtype='datetime64')
print("Date Array:", date_array)

# Calculate the difference in days
date_diff = np.diff(date_array)
print("Difference in Days:", date_diff)
```

NumPy String

Definition: Functions to perform operations on string arrays.

Example 1:

```
arr = np.array(['a', 'b', 'c'])
print(np.char.upper(arr))
```

Example 2:

```
import numpy as np

# Create an array of strings
string_array = np.array(['hello', 'world'])

# Convert to uppercase
uppercase_array = np.char.upper(string_array)
print("Uppercase Strings:", uppercase_array)

# Concatenate strings
concat_string = np.char.add(string_array[0], string_array[1])
print("Concatenated String:", concat_string)
```


Projects and Exercises

1. Project 1: Data Analysis with NumPy

- Description: Analyze a dataset (e.g., CSV file) using NumPy to compute summary statistics and perform basic data manipulations.
- Objectives: Load data, compute mean, median, standard deviation, filter data, etc.

2. Exercise 2: Array Operations

- Task: Create various NumPy arrays and perform basic operations such as slicing, indexing, and reshaping.

3. Exercise 3: Linear Algebra Applications

- Task: Solve a system of linear equations using NumPy.

4. Exercise 4: Random Number Generation

- Task: Generate random numbers for a given distribution and visualize the results using a histogram.

5. Exercise 5: Sorting and Searching

- Task: Implement searching and sorting algorithms on NumPy arrays and compare their performance with Python's built-in functions.

6. Exercise 6: Mathematical Computations

- Task: Perform a series of mathematical operations on NumPy arrays and compare the results with manual calculations.

References:

- <https://www.geeksforgeeks.org/introduction-to-numpy/>
- <https://www.w3schools.com/python/numpy/default.asp>
- <https://www.javatpoint.com/numpy-tutorial>
- <https://www.mygreatlearning.com/blog/python-numpy-tutorial/#data-type-objects-dtype>

Introduction to Pandas

What is Pandas?

Pandas is a Python library used for working with data sets. It has functions for analyzing, cleaning, exploring, and manipulating data.

The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008.

Why Use Pandas?

Pandas allows us to analyze big data and make conclusions based on statistical theories. Pandas can clean messy data sets, and make them readable and relevant. Relevant data is very important in data science.

What Can Pandas Do?

Pandas gives you answers about the data like is there a correlation between two or more columns?

What is average value?

Max value?

Min value?

Pandas are also able to delete rows that are not relevant, or contains wrong values, like empty or NULL values. This is called cleaning the data.

Pandas Getting Started

Installation of Pandas

- Python: Ensure you have Python 3.6 or later installed. You can download it from the official Python website (<https://www.python.org/>).
- Additionally, you can also install the PyCharm IDE ([Installation Guide](#)).
- If you have Python and PIP already installed on a system, then installation of Pandas is very easy.

Install it using this command:

```
C:\Users\Your Name>pip install pandas
```

Import Pandas

Once Pandas is installed, import it in your applications by adding the import keyword:
import pandas

Now Pandas is imported and ready to use.

Example 1: Get your own Python Server

```
import pandas

mydataset = {
    'cars': ["BMW", "Volvo", "Ford"],
    'passings': [3, 7, 2]
}

myvar = pandas.DataFrame(mydataset)

print(myvar)
```

Pandas as Pd

Pandas **isare** usually imported under the pd alias.

alias: In Python alias are an alternate name for referring to the same thing.

Create an alias with the as keyword while importing:

```
import pandas as pd
```

Now the Pandas package can be referred to as pd instead of pandas.

Example 1:

```
import pandas as pd

mydataset = {
    'cars': ["BMW", "Volvo", "Ford"],
    'passings': [3, 7, 2]
}

myvar = pd.DataFrame(mydataset)

print(myvar)
```

Checking Pandas Version

The version string is stored under __version__ attribute.

Example 1:

```
import pandas as pd

print(pd.__version__)
```

Pandas Series

What is a Series?

A Pandas Series is like a column in a table. It is a one-dimensional array holding data of any type.

Example 1: Create a simple Pandas Series from a list

```
import pandas as pd

a = [1, 7, 2]

myvar = pd.Series(a)

print(myvar)
```

Labels

If nothing else is specified, the values are labeled with their index number. First value has index 0, second value has index 1 etc. This label can be used to access a specified value.

Example 1: Return the first value of the Series

```
print(myvar[0])
```

Create Labels

With the index argument, you can name your own labels.

Example 1: Create your own labels

```
import pandas as pd

a = [1, 7, 2]

myvar = pd.Series(a, index = ["x", "y", "z"])

print(myvar)
```

Key/Value Objects as Series

You can also use a key/value object, like a dictionary, when creating a Series.

Example 1: Create a simple Pandas Series from a dictionary

```
import pandas as pd

calories = {"day1": 420, "day2": 380, "day3": 390}

myvar = pd.Series(calories)

print(myvar)
```

To select only some of the items in the dictionary, use the index argument and specify only the items you want to include in the Series.

Example 2: Create a Series using only data from "day1" and "day2"

```
import pandas as pd

calories = {"day1": 420, "day2": 380, "day3": 390}

myvar = pd.Series(calories, index = ["day1", "day2"])

print(myvar)
```

Data Frames

Data sets in Pandas are usually multi-dimensional tables, called Data Frames. Series is like a column, a Data Frame is the whole table.

Example 1: Create a Data Frame from two Series

```
import pandas as pd

data = {

    "calories": [420, 380, 390],

    "duration": [50, 40, 45]

}

myvar = pd.DataFrame(data)

print(myvar)
```

Pandas Data Frames

What is a Data Frame?

A Pandas Data Frame is a 2-dimensional data structure, like a 2-dimensional array, or a table with rows and columns.

Example 1: Create a simple Pandas Data Frame

```
import pandas as pd

data = {

    "calories": [420, 380, 390],

    "duration": [50, 40, 45]

}
```

```
#load data into a Data Frame object:
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

Locate Row

As you can see from the result above, the Data Frame is like a table with rows and columns. Pandas use the loc attribute to return one or more specified row(s)

Example 1: Return row 0

```
#refer to the row index:
```

```
print(df.loc[0])
```

Named Indexes

With the index argument, you can name your own indexes.

Example 1: Add a list of names to give each row a name

```
import pandas as pd
```

```
data = {
```

```
    "calories": [420, 380, 390],
```

```
    "duration": [50, 40, 45]
```

```
}
```

```
df = pd.DataFrame(data, index = ["day1", "day2", "day3"])
```

```
print(df)
```

Locate Named Indexes

Use the named index in the loc attribute to return the specified row(s).

Example 1: Return "day2"

```
#refer to the named index:
```

```
print(df.loc["day2"])
```

Pandas Read CSV

Read CSV Files

A simple way to store big data sets is to use CSV files (comma separated files). CSV files contain plain text and is a well known format that can be read by everyone including Pandas. In our examples we will be using a CSV file called 'data.csv'. Open data.csv. or Open data.csv

Example 1: Load the CSV into a Data Frame

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.to_string())
```

Pandas - Analyzing Data Frames

Viewing the Data

One of the most used methods for getting a quick overview of the Data Frame, is the `head()` method. The `head()` method returns the headers and a specified number of rows, starting from the top.

Example 1: Get a quick overview by printing the first 10 rows of the Data Frame

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.head(10))
```

Info About the Data

The `DataFrame` object has a method called `info()`, that gives you more information about the data set.

Example 1: Print information about the data

```
print(df.info())
```

Null Values

The `info()` method also tells us how many Non-Null values there are present in each column, and in our data set it seems like there are 164 of 169 Non-Null values in the "Calories" column. Which means that there are 5 rows with no value at all, in the "Calories" column, for whatever reason. Empty values, or Null values, can be bad

Commented [NJ1]:

when analyzing data, and you should consider removing rows with empty values. This is a step towards what is called cleaning data.

Dealing with Rows and Columns

A Data frame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns. We can perform basic operations on rows/columns like selecting, deleting, adding, and renaming.

Column Selection: In Order to select a column in Pandas Data Frame, we can either access the columns by calling them by their columns name.

Example 1:

```
# Import pandas package
import pandas as pd

# Define a dictionary containing employee data
data = {'Name': ['Jai', 'Princi', 'Gaurav', 'Anuj'],
        'Age': [27, 24, 22, 32],
        'Address': ['Delhi', 'Kanpur', 'Allahabad', 'Kannauj'],
        'Qualification': ['Msc', 'MA', 'MCA', 'Phd']}

# Convert the dictionary into DataFrame
df = pd.DataFrame(data)

# select two columns
print(df[['Name', 'Qualification']])

# importing pandas package
import pandas as pd

# making data frame from csv file
data = pd.read_csv("nba.csv", index_col="Name")

# retrieving row by loc method
first = data.loc["Avery Bradley"]

second = data.loc["R.J. Hunter"]

print(first, "\n\n", second)
```

Indexing and Selecting Data

Indexing in pandas means simply selecting particular rows and columns of data from a Data Frame. Indexing could mean selecting all the rows and some of the columns,

some of the rows and all of the columns, or some of each of the rows and columns. Indexing can also be known as Subset Selection.

Indexing a Data frame using indexing operator `[]`: Indexing operator is used to refer to the square brackets following an object. The `.loc` and `.iloc` indexers also use the indexing operator to make selections. In this indexing operator to refer to `df[ ]`.

Example 1: Selecting a single column

In order to select a single column, we simply put the name of the column in-between the brackets

```
# importing pandas package
import pandas as pd

# making data frame from csv file
data = pd.read_csv("nba.csv", index_col="Name")

# retrieving columns by indexing operator
first = data["Age"]

print(first)
```

Example 2: Selecting a single row

In order to select a single row using `.loc[]`, we put a single row label in a `.loc` function.

```
# importing pandas package
import pandas as pd

# making data frame from csv file
data = pd.read_csv("nba.csv", index_col="Name")

# retrieving row by loc method
first = data.loc["Avery Bradley"]
second = data.loc["R.J. Hunter"]

print(first, "\n\n", second)
```

Example 3: Selecting a single row

In order to select a single row using `.iloc[]`, we can pass a single integer to `.iloc[]` function.

```
import pandas as pd

# making data frame from csv file

data = pd.read_csv("nba.csv", index_col="Name")

# retrieving rows by iloc method

row2 = data.iloc[3]

print(row2)
```

Working with missing Data

Missing Data can occur when no information is provided for one or more items or for a whole unit. Missing Data is a very big problem in ~~real-real~~ life scenario. Missing Data can also refer to as NA(Not Available) values in pandas.

Checking for missing values using `isnull()` and `notnull()`: In order to check missing values in Pandas DataFrame, we use a function `isnull()` and `notnull()`. Both functions help in checking whether a value is NaN or not. These functions can also be used in Pandas Series ~~in-order~~ to find null values in a series.

Example 1:

```
importing pandas as pd
import pandas as pd

# importing numpy as np
import numpy as np

# dictionary of lists
dict = {'First Score':[100, 90, np.nan, 95],
        'Second Score': [30, 45, 56, np.nan],
        'Third Score':[np.nan, 40, 80, 98]}

# creating a dataframe from list
df = pd.DataFrame(dict)

# using isnull() function
df.isnull()
```

Filling missing values using `fillna()`, `replace()` and `interpolate()`: In order to fill null values in a datasets, we use `fillna()`, `replace()` and `interpolate()` functions. These functions help in filling a null values in datasets of a Data Frame. `Interpolate()` function is ~~basically~~ used to fill NA values in the data Frame but it uses various interpolation technique to fill the missing values rather than hard-coding the value.

Example 1:

```
# importing pandas as pd
import pandas as pd

# importing numpy as np
import numpy as np

# dictionary of lists
dict = {'First Score':[100, 90, np.nan, 95],
        'Second Score': [30, 45, 56, np.nan],
        'Third Score':[np.nan, 40, 80, 98]}

# creating a dataframe from dictionary
df = pd.DataFrame(dict)

# filling missing value using fillna()
df.fillna(0)
```

Iterating over Rows and Columns

Iteration is a general term for taking each item of something, one after another. Pandas Data Frame consists of rows and columns so, ~~in-order~~ to iterate over data frame, we have to iterate a data frame like a dictionary.

Iterating over rows: ~~In-order~~ To iterate over rows, we can use three functions iteritems(), iterrows(), itertuples(). These three functions will help in iteration over rows.

Example 1:

```
# importing pandas as pd
import pandas as pd

# dictionary of lists
dict = {'name':["aparna", "pankaj", "sudhir", "Geeku"],
        'degree': ["MBA", "BCA", "M.Tech", "MBA"],
        'score':[90, 40, 80, 98]}

# creating a dataframe from a dictionary
df = pd.DataFrame(dict)

print(df)
```

Example 2: Now we apply iterrows() function in order to get an each element of rows.

```
# importing pandas as pd
import pandas as pd

# dictionary of lists
dict = {'name':["aparna", "pankaj", "sudhir", "Geeku"],
        'degree': ["MBA", "BCA", "M.Tech", "MBA"],
```

```
'score':[90, 40, 80, 98]}

# creating a dataframe from a dictionary
df = pd.DataFrame(dict)

# iterating over rows using iterrows() function
for i, j in df.iterrows():
    print(i, j)
    print()
```

Iterating over Columns: In order to iterate over columns, we need to create a list of data frame columns and then iterating through that list to pull out the data frame columns.

Example 3:

```
# importing pandas as pd
import pandas as pd

# dictionary of lists
dict = {'name':['aparna', "pankaj", "sudhir", "Geeku"],
        'degree': ["MBA", "BCA", "M.Tech", "MBA"],
        'score':[90, 40, 80, 98]}

# creating a dataframe from a dictionary
df = pd.DataFrame(dict)

print(df)
```

References

- https://www.w3schools.com/python/pandas/pandas_exercises.asp
- <https://www.geeksforgeeks.org/python-pandas-dataframe/>
- <https://www.w3resource.com/python-exercises/pandas/plotting/index.php>